HSBox3 CTF – Cybersecurity Assessment Report

TrustNet Secure – Cybersecurity Internship Challenge

1. Executive Summary

This report explains, in plain language, how a test website called **HSBox3** (hosted at `10.213.41.211`) was examined for security weaknesses as part of a Capture-The-Flag (CTF) training exercise. A CTF is a safe, legal practice exercise where testers hunt for hidden "flags" (secret codes) that prove a weakness was found and exploited.

Two flags were successfully captured:

Flag	Value
Flag 1	`Flag1{77629843d5f2dc9ccccb3bee87fa2992}`
Flag 2	`Flag2{b39fb742a90fbd35111c77d6cd982fe7}`

Both flags were found because the website **leaked information it should have kept hidden** — one inside the raw HTML code of a page, and one inside a hidden website (reachable only by a special trick) running WordPress. Along the way, the test also uncovered an **exposed password file** readable by anyone, which is a serious real-world risk even though it was only a training file here.

2. Scope and Objective

- **Target:** `10.213.41.211` (nicknamed "hackerschool.local")
- **Goal:** Identify weaknesses an attacker could use, and retrieve the two hidden "flags" placed on the system as proof of successful exploitation.
- **Type of test:** Black-box web application assessment (the tester started with no inside knowledge, just the target's IP address).

3. Methodology

The assessment followed the standard steps a real attacker — or a professional penetration tester — would follow:

1. **Reconnaissance (information gathering):** Scan the target to see what services are running and what doors are open.
2. **Enumeration:** Dig deeper into the service that was found (a website) to see what pages, folders, and files exist.
3. **Analysis of exposed files:** Read any files the scan uncovered that shouldn't normally be public.
4. **Directory brute-forcing:** Use a tool to automatically guess hidden folder names on the website.
5. **Virtual host discovery:** Check whether the same server was secretly hosting a *second*, different website under a different name.
6. **Flag extraction:** Read page source code and server responses to pull out the hidden flags.

4. Tools Used

Tool	What it does (plain English)
Nmap	Scans a computer to find which "doors" (network ports) are open and what software is running behind them.
Web browser (view-source)	Lets you see the hidden code behind a webpage, which can reveal secrets the page doesn't show visually.
Gobuster	Automatically tries thousands of common folder/file names against a website to discover hidden pages.

| | **curl** | A command-line tool to request web pages directly, including tricking the server into showing a *different* hidden site using a custom "Host" header. | | **robots.txt analysis** | Reading a website's "robots.txt" file, which is meant to tell search engines what *not* to index — but often accidentally reveals sensitive folder names to humans too. |

5. Step-by-Step Findings

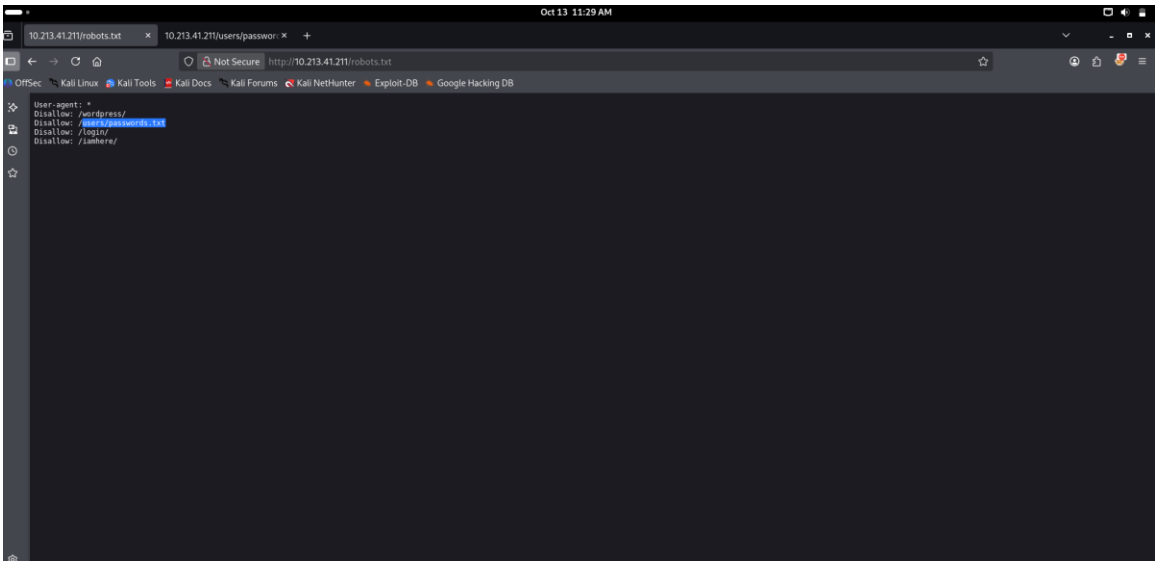
Step 1 — Scanning the target (Nmap)

An Nmap scan of `10.213.41.211` showed one open port: **port 80 (web server)**, running **Apache 2.4.7 on Ubuntu**. Nmap's built-in web script also immediately flagged a `robots.txt` file listing four "disallowed" (supposedly hidden) paths:

- `/wordpress/`
- `/users/passwords.txt`
- `/login/`
- `/iamhere/`

Why this matters: `robots.txt` is only a polite request to search engines — it is **not** a security control. Anyone, including an attacker, can read it and go straight to the folders it lists.





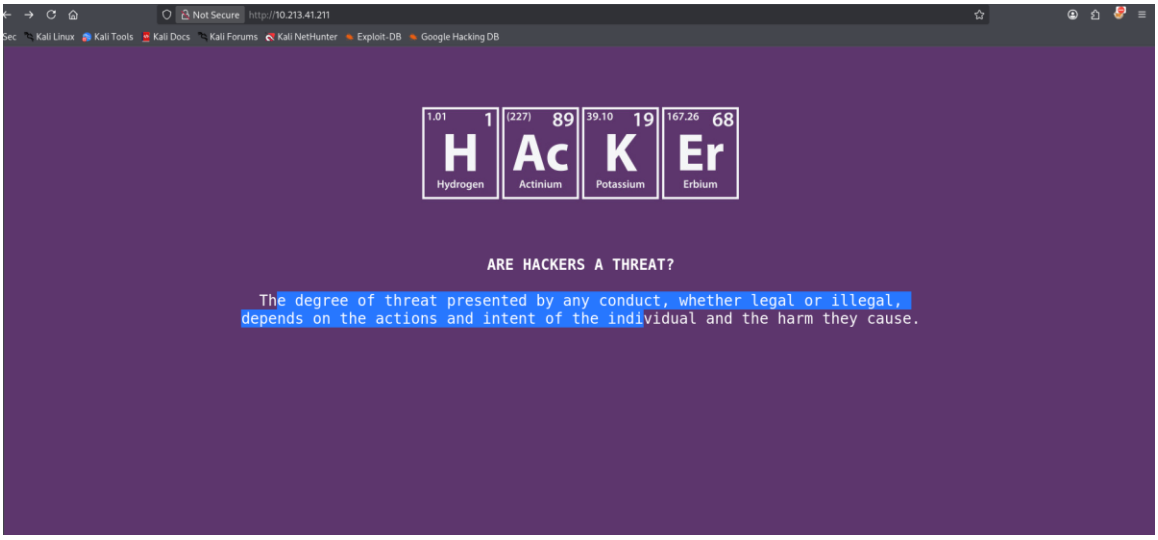
Step 2 — Finding Flag 1 in the page source code

Visiting the website showed a page asking "Are Hackers a Threat?" with a periodic-table-style graphic spelling out **H-Ac-K-Er**. Viewing the page's underlying HTML source code revealed a line that was never meant to be seen by normal visitors:

```
<p style="font-family: verdana; font-size: 120%; color: #5D366D;">
Flag1{77629843d5f2dc9ccccb3bee87fa2992}
</p>
```

The flag text was styled in a colour that blended into the page background, so it was invisible on screen but fully readable in the source code — a classic case of "security by hiding," which does not actually secure anything.

✅ **Flag 1 captured:** `Flag1{77629843d5f2dc9ccccb3bee87fa2992}`



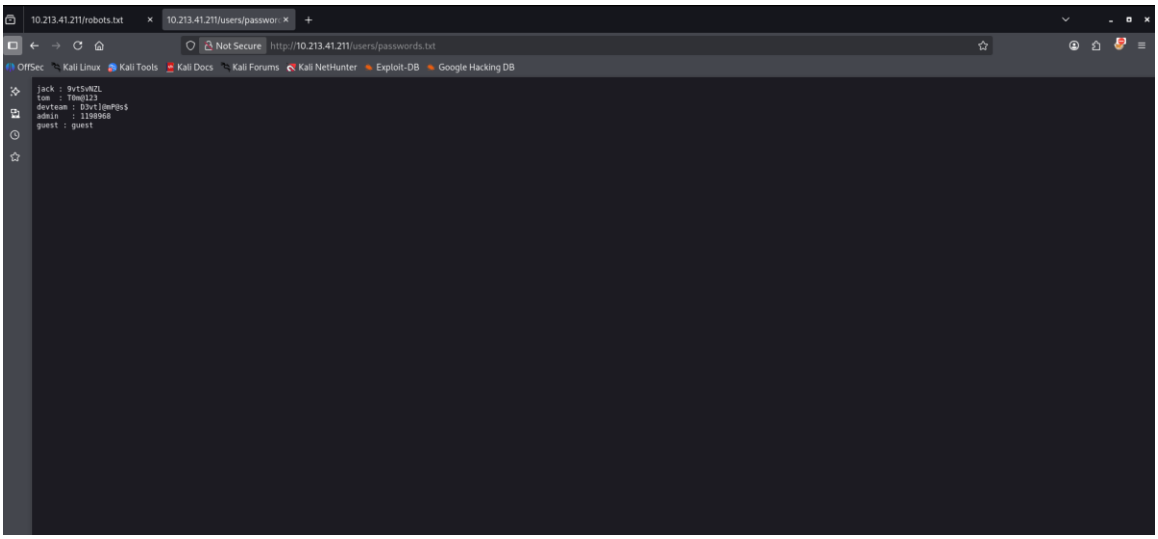


Step 3 — Reading the leaked password file

Following the `robots.txt` hint to `/users/passwords.txt` revealed a **plain-text list of usernames and passwords**:

```
jack     : 9vtSvNZL
tom      : T0m@123
devteam  : D3vt]@mP@s$
admin    : 1198968
guest    : guest
```

Why this matters: Storing credentials in an unprotected, publicly reachable text file is one of the most dangerous mistakes a website can make. In a real environment this would allow instant account takeover, including of an **admin** account.



Step 4 — Discovering hidden folders (Gobuster)

A directory brute-force scan with Gobuster (using a common wordlist) confirmed three real folders on the server:

- `/img/`
- `/users/`
- `/flags/`

This independently confirmed and expanded on what `robots.txt` had hinted at, showing the site had multiple areas not linked from its visible pages.

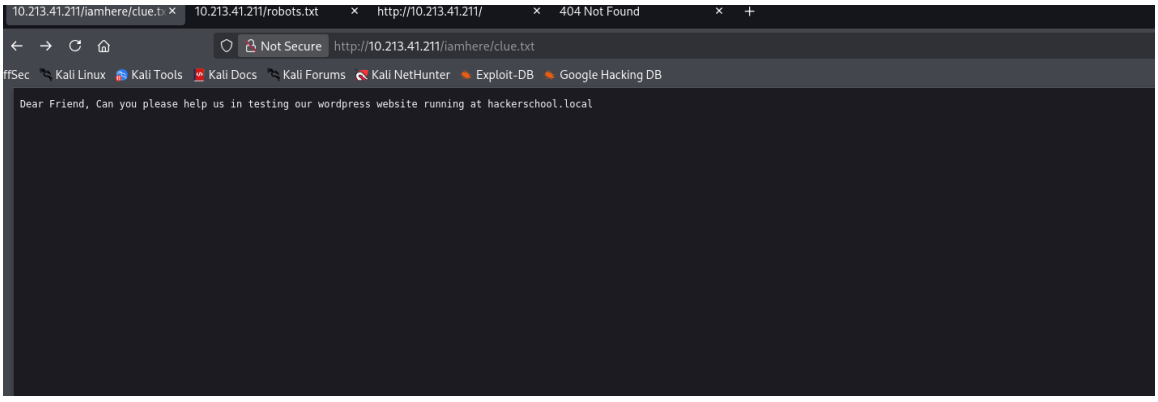


Step 5 — Following the clue to a hidden website

The `/iamhere/clue.txt` file (also listed in `robots.txt`) contained a direct hint:

> *"Dear Friend, can you please help us in testing our WordPress website running at hackerschool.local"*

This told the tester that the same server was secretly hosting a **second website** — a WordPress site — that only responds if you specifically ask for it by name (`hackerschool.local`) instead of by IP address. This is called **virtual hosting**, and it's a common way organizations run several websites on one server.



Step 6 — Finding Flag 2 on the hidden WordPress site

Using `curl` with a custom "Host" header to request the site *as if* the browser were asking for `hackerschool.local`, the hidden WordPress site appeared. Its page title contained the second flag directly:

```
<title>HSBox3 – Flag2{b39fb742a90fbd35111c77d6cd982fe7}</title>
```

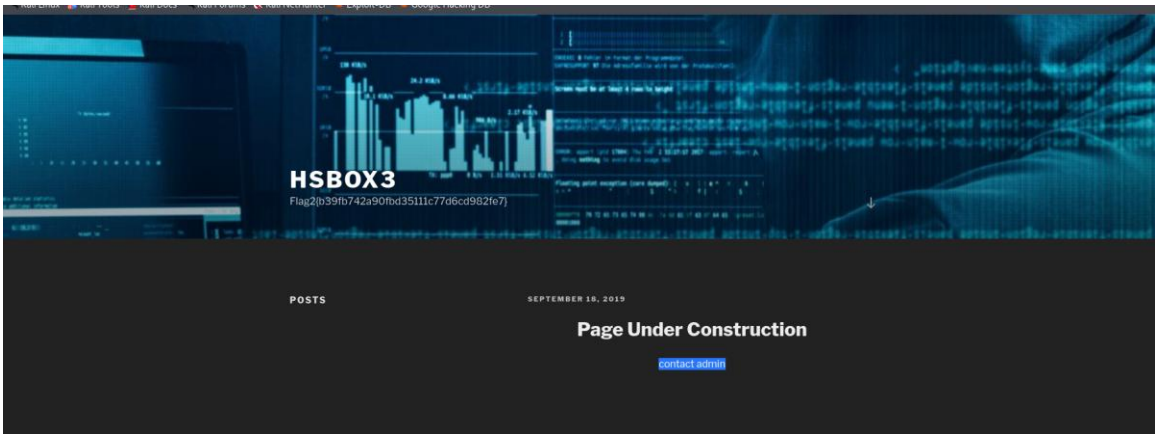
Visiting the site normally in a browser (after pointing `hackerschool.local` to the target IP) confirmed the same flag displayed on the homepage banner, alongside an "under construction" WordPress page and a "contact admin" link.

✅ **Flag 2 captured:** `Flag2{b39fb742a90fbd35111c77d6cd982fe7}`

```
┌──(naruto㉿kali)-[~]
└─$ curl -i -H 'Host: hackerschool.local' http://10.213.41.211/
HTTP/1.1 200 OK
Date: Tue, 13 Oct 2026 16:23:04 GMT
Server: Apache/2.4.7 (Ubuntu)
X-Powered-By: PHP/5.5.9-1ubuntu4.29
Link: <http://hackerschool.local/?rest_route=/>; rel="https://api.w.org/"
Vary: Accept-Encoding
Transfer-Encoding: chunked
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="en-US" class="no-js no-svg">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="profile" href="http://gmpg.org/xfn/11">

<script>(function(html){html.className = html.className.replace(/\bno-js\b/,'js')})(document.documentElement);</script>
<title>HSBox3 &#8211; Flag2{b39fb742a90fbd35111c77d6cd982fe7}</title>
```



6. Supporting Evidence

All findings above are backed by screenshots captured during testing, including:

- The Nmap scan output showing the open port and `robots.txt` disallow list
- The browser view of the "Are Hackers a Threat?" page and its HTML source containing Flag 1
- The `robots.txt` file contents
- The exposed `passwords.txt` file contents
- The Gobuster directory brute-force results
- The `/iamhere/clue.txt` hint about the WordPress site
- The `curl` request/response revealing Flag 2 in the page title
- The live WordPress homepage displaying Flag 2

(Original screenshots are included in the uploaded Word document.)

7. Summary of Weaknesses Found

#	Weakness	Risk Level	Explanation

#	Weakness	Risk Level	Explanation
1	Sensitive paths listed in `robots.txt`	Medium	Tells attackers exactly where to look instead of hiding anything.
2	Secret text hidden only by matching page colour (not removed from code)	Medium	Anyone who views page source can read "hidden" information.
3	Plain-text password file publicly accessible	**Critical**	Full set of usernames/passwords exposed with no protection at all.
4	Predictable/guessable hidden directories (`/img/`, `/users/`, `/flags/`)	Low–Medium	Confirms that "security through obscurity" (hiding folders instead of locking them) is not effective.
5	Hidden virtual host (WordPress site) discoverable via a simple clue file	Medium	Shows a second, less-monitored website existed without clear access control.

8. Recommended Remediation

1. **Remove credentials from web-accessible locations immediately.** Never store usernames/passwords in a plain `.txt` file inside a public web folder. Use a proper authentication system (e.g., a database with hashed/salted passwords) and store any required secrets outside the web root.
2. **Change the leaked passwords.** All accounts shown in `passwords.txt` (jack, tom, devteam, admin, guest) should have their passwords reset immediately, since they must now be treated as compromised.
3. **Stop relying on `robots.txt` for security.** Use it only for its intended purpose (guiding search engines). Protect truly sensitive folders with proper authentication and server-side access controls (e.g., `.htaccess`, login walls, or firewall rules).
4. **Remove hidden content from page source rather than disguising it with colour.** If a flag, comment, or note isn't meant for users, delete it from the HTML entirely rather than just making it visually invisible.
5. **Apply least-privilege directory permissions.** Disable directory listing and ensure folders like `/img/`, `/users/`, and `/flags/` cannot be browsed or guessed into revealing content.
6. **Secure or remove the WordPress installation.** If `hackerschool.local` is a real, needed service, it should require authentication, be patched/updated, and not be discoverable through an open

clue file. If it was only for internal testing, it should be taken offline or isolated from the public-facing server.

7. **Conduct regular security scanning.** Running tools like Nmap and Gobuster against your own systems periodically ("self-assessment") helps catch these issues before an outside attacker does.

9. Conclusion

This exercise demonstrated how a handful of small, individually "minor-looking" mistakes — a readable `robots.txt`, an un-deleted code comment, a forgotten text file, and an undisclosed second website — can be chained together to fully compromise a target. Both challenge flags were successfully retrieved, and in the process a critical real-world risk (a plain-text password file) was identified. The remediation steps above would close every weakness found during this assessment.